

The Claude Code Workflow: Explore, Plan, Code, Commit

If you take one thing away from this course, let it be this workflow. Most people jump straight to asking Claude to write code - which means more course-correcting later. This four-step loop avoids that.

Explore · Plan · Code · Commit

IN THIS LESSON

Work through the four-step loop that makes Claude Code reliable:

- **Explore & Plan** in Plan Mode - let Claude read the codebase and propose an approach before any code is written
- **Code** against an approved plan - with clear success criteria, the right tools, and a test suite
- **Commit** with confidence - a read-only subagent review, then a commit message in your style
- The habit of **course-correcting early**, while it's cheapest

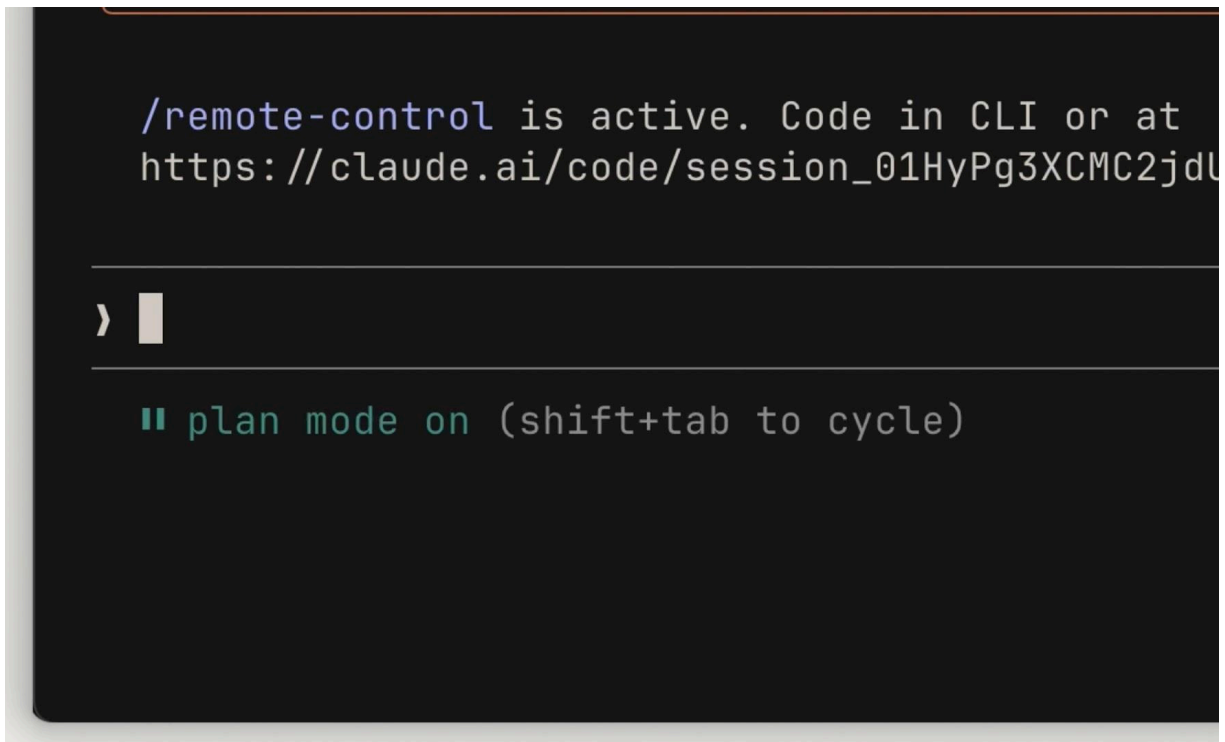
ON THIS PAGE

- › 1. Explore & Plan
- › 2. Code
- › 3. Commit
- › 4. Recap

01 Explore & Plan

The fastest way to handle the first two steps is with **Plan Mode**. In Plan Mode, Claude can't edit files - it just reads them to gather information about how it will tackle the implementation.

To enter Plan Mode, press `Shift + Tab` until you see “Plan Mode” under the text input. Then write a prompt like the one below.

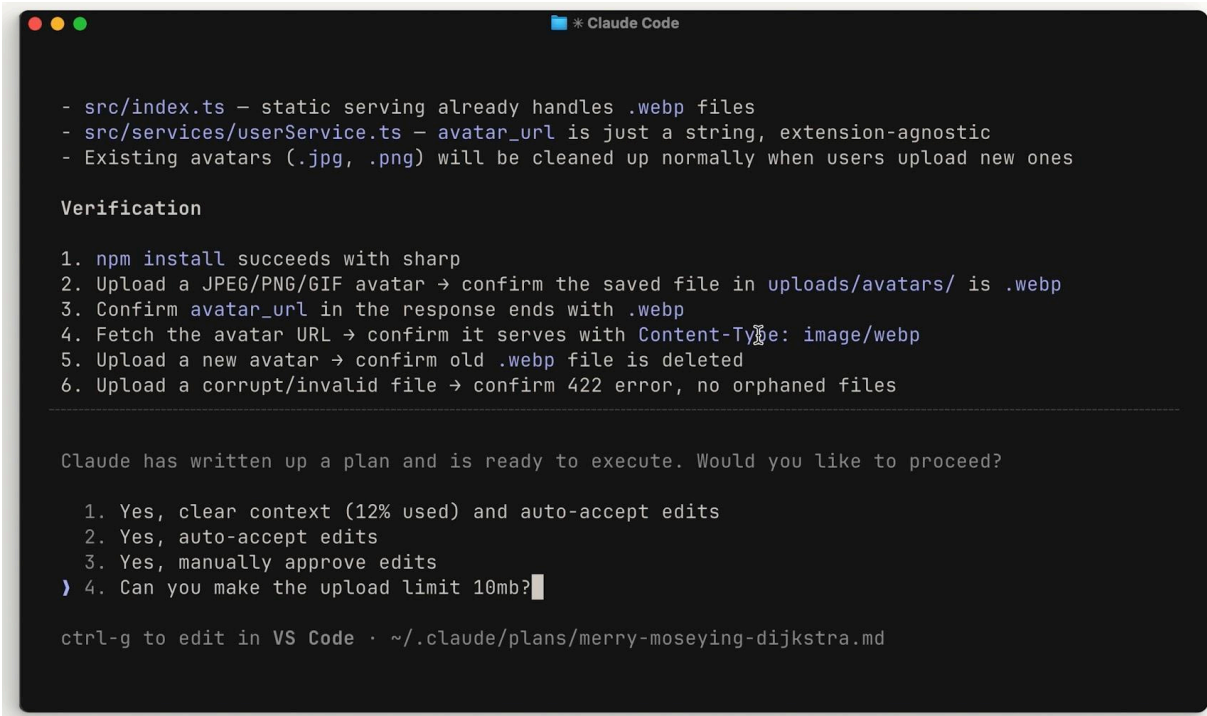


Press Shift + Tab until the footer reads “plan mode on.”

TRY A PROMPT LIKE THIS

› I need to add WebP conversion to our image upload pipeline. Figure out where in the pipeline it should happen, whether we need new dependencies, and how to approach it.

Claude reads the relevant files, runs some web searches, and gives you a plan of action. Review it and decide if it meets your criteria - if not, ask it to revise specific areas.



```
* Claude Code

- src/index.ts - static serving already handles .webp files
- src/services/userService.ts - avatar_url is just a string, extension-agnostic
- Existing avatars (.jpg, .png) will be cleaned up normally when users upload new ones

Verification

1. npm install succeeds with sharp
2. Upload a JPEG/PNG/GIF avatar → confirm the saved file in uploads/avatars/ is .webp
3. Confirm avatar_url in the response ends with .webp
4. Fetch the avatar URL → confirm it serves with Content-Type: image/webp
5. Upload a new avatar → confirm old .webp file is deleted
6. Upload a corrupt/invalid file → confirm 422 error, no orphaned files

Claude has written up a plan and is ready to execute. Would you like to proceed?

1. Yes, clear context (12% used) and auto-accept edits
2. Yes, auto-accept edits
3. Yes, manually approve edits
4. Can you make the upload limit 10mb?

ctrl-g to edit in VS Code · ~/.claude/plans/merry-moseying-dijkstra.md
```

Claude returns a plan and asks how to proceed - review it, or request revisions before any code is written.

COURSE-CORRECT HERE FIRST

This is the best place to course-correct - it's before any code is written. You can also run the explore subagent outside Plan Mode when you just want a general summary of your codebase without intending to make changes.

02 Code

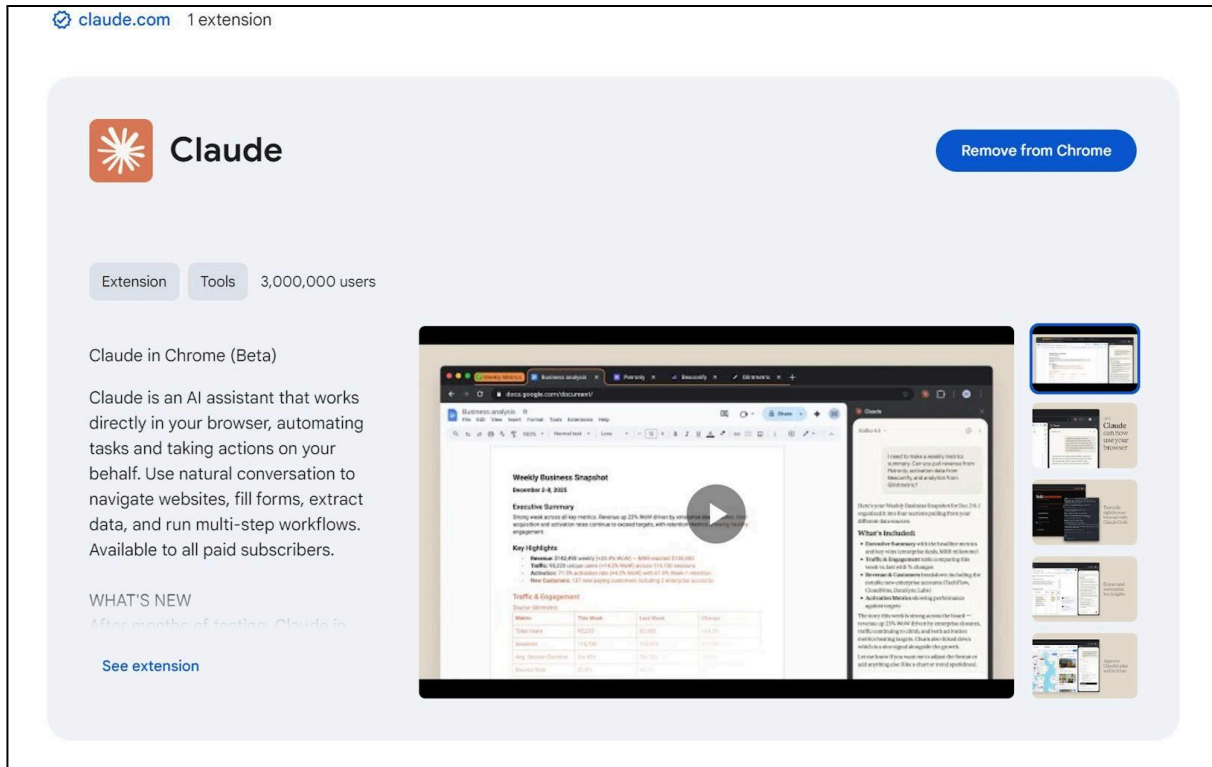
Once the plan looks good, select **Approve** to accept it and let Claude work through the list items. You can choose whether Claude auto-accepts file edits or asks you each time.

Claude does its best to troubleshoot before considering the plan “finished,” but at times you’ll need to step in. That’s the benefit of Plan Mode - after execution you also have the context of *how* you got to the results, which helps guide Claude’s next decisions.

A few tips to make the coding phase smoother:

- **Define success criteria.** For Claude to be confident in its results, it needs to be clear on what “correct” looks like. Make this explicit when writing your plan.

- **Add tools.** Tools that help Claude reach its goals remove a lot of back-and-forth. For web UIs, install the Claude in Chrome extension so Claude Code can drive a browser tab and test the UI directly.
- **Include a test suite.** Give Claude a test suite to validate against - it can even write the tests for you. Make sure they're a reliable source of truth first, to avoid false positives.



Adding tools - like the Claude in Chrome extension - lets Claude test its own work and cut the back-and-forth.

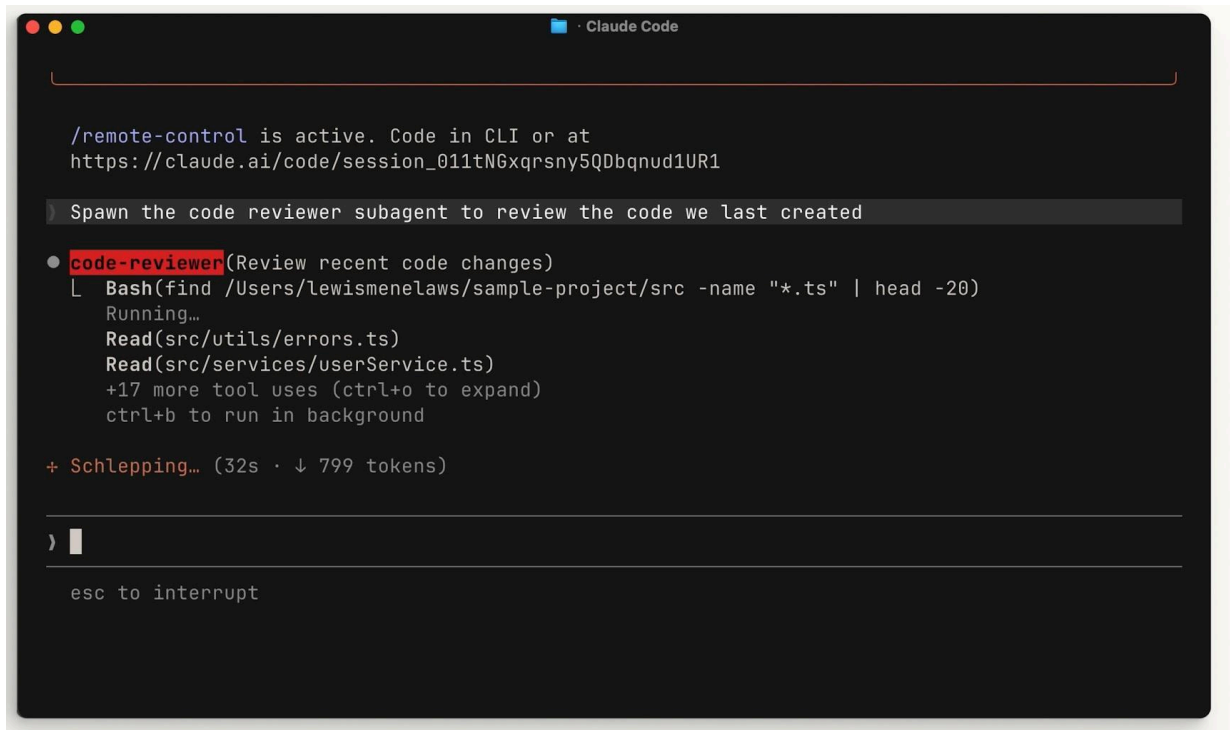
QUICK TIP

If Claude keeps running into the same issues, ask it to save the solution to its `CLAUDE.md` file - so it doesn't rediscover the fix next time.

03 Commit

Once you've tested the changes yourself and are happy with the results, it's time to push. But before you commit, run a **subagent code reviewer** over your work. A subagent gets a fresh

pair of eyes on the codebase - it doesn't carry the bias the main agent picked up while writing the code this session.



Spawn a read-only code-reviewer subagent for a fresh-eyes pass before you commit.

Then get Claude to generate a commit message in your style. Rinse and repeat.

04 Recap

To be effective with Claude Code, follow the Explore → Plan → Code → Commit workflow:

Step	What it does
Explore	Gives Claude the relevant context it needs for your project.
Plan	Creates a plan of action that Claude uses to measure success.
Code	The back-and-forth between you and Claude before settling on the final outcome.
Commit	Helps you review and push your code, so you can start the next feature.

Explore → Plan → Code → Commit - the loop that keeps Claude on track.

Hands-On Workshop: The Claude Code Workflow

Explore → Plan → Code → Commit

In about 15 minutes you'll take a real feature - adding WebP conversion to a camera snapshot pipeline - through Explore, Plan, Code, and Commit, course-correcting early while it's cheapest.

All examples use TypeScript.

Plan Mode · success criteria · subagent review

IN THIS DEMO

- Let Claude read the code and propose a plan before writing anything.
- Approve the plan and let Claude implement it against a test suite.
- Get a fresh-eyes review from a subagent, then a commit message in our style.

ON THIS PAGE

- › 1. Why This Loop
- › 2. Prerequisites
- › 3. Explore and Plan (in Plan Mode)
- › 4. Code (Against the Plan)
- › 5. Commit (with a Fresh-Eyes Review)
- › 6. Summary
- › 7. Troubleshooting

01 Why This Loop

Most people jump straight to “write the code,” which means more correcting later. The Explore→Plan→Code→Commit loop front-loads the thinking, so problems get caught before any code exists - where they are cheapest to fix.

Step	What it does
Explore	Gives Claude the context it needs about your project.
Plan	Produces an approach you can review and measure success against.
Code	The back-and-forth to reach the final implementation.
Commit	Review and push, then start the next feature.

02 Prerequisites

- Claude Code installed and running (`claude` is available in your terminal).
- The `demo-5-workflow` folder from the workshop bundle. Open it:

```
cd demo-5-workflow
claude
```

WHAT'S IN THE FOLDER

This folder contains a small snapshot pipeline, a test suite, a `CLAUDE.md`, and a read-only code-reviewer subagent - everything the loop needs.

Project Files - [Link](#)

03 Explore and Plan (in Plan Mode)

Plan Mode lets Claude read the code but **not edit it**, so it can propose an approach safely.

Enter Plan Mode by pressing **Shift + Tab** until the footer reads “plan mode on”. Then prompt:

```
I need to add WebP conversion to our snapshot pipeline. Figure out where
in the pipeline it should happen, whether we need a new dependency, and
```



```
how to approach it. Keep all endpoints TLS-only per our conventions.
```

Claude reads `src/pipeline/`, `src/api/`, and the tests, then returns a plan - likely proposing the conversion inside `src/pipeline/imagePipeline.ts`, where raw frames are already processed.

This is the best place to course-correct. Review the plan. If anything is off - wrong location, missing a step, no test coverage - ask for a revision now, before any code is written:

```
Revise: keep the original frame too, and add a test that asserts the  
output is valid WebP.
```

04 Code (Against the Plan)

When the plan looks right, **Approve** it. Claude works through the steps. You can let it auto-accept edits or confirm each one.

Two things make this phase smoother, and both are already set up in this demo:

- **Success criteria.** The plan should state what “correct” looks like. Here it is concrete: the pipeline returns WebP bytes, the original frame is preserved, and the tests pass.
- **A test suite as the source of truth.** Run it after the change:

```
npm test
```

`tests/imagePipeline.test.ts` validates the conversion. If a test fails, Claude troubleshoots against it rather than guessing. If Claude hits the same issue repeatedly, ask it to save the fix to `CLAUDE.md` so it is not rediscovered next time.

05 Commit (with a Fresh-Eyes Review)

Before committing, get a second opinion from a subagent. It runs in its own context, so it does not carry the bias the main agent picked up while writing the code this session.

```
Use the code-reviewer subagent to review the WebP changes before I commit.
```

The reviewer reads the changed files and returns a structured assessment - security, correctness, and whether it is ready to merge. Address anything it flags.

Then have Claude write the commit message in your style:

```
Write a commit message for these changes in our conventional-commits style.
```

Review, commit, and you are ready to start the next feature. Rinse and repeat.

06 Summary

- Explore and Plan in Plan Mode (**Shift + Tab**) so Claude proposes an approach before editing anything.
- Course-correct on the plan - it is the cheapest place to fix direction.
- Code against clear success criteria and a reliable test suite.
- Commit only after a read-only subagent review, then generate a commit message in your style.
- This loop ties the other lessons together: Plan Mode (explore), `CLAUDE.md` (conventions), subagents (review), and context discipline throughout.

07 Troubleshooting

Symptom	Resolution
Claude starts editing before you are ready	You are not in Plan Mode. Press Shift + Tab until the footer reads “plan mode on”.
The plan misses requirements	Add explicit success criteria to your prompt and ask for a revision before approving.
Tests pass but the result is wrong	The tests are not a reliable source of truth yet. Tighten them first, then re-run.
The reviewer repeats the main agent’s blind spots	Confirm it is a separate subagent with read-only tools, not the same thread reviewing itself.

Front-load the thinking - Explore, Plan, Code, Commit - and catch problems before any code exists.